Geração Automática de Resumos e de Conhecimentos em Mapas Mentais Utilizando LLM e Python

Gabriel Bevilacqua Barros
Curso de Bacharelado em Ciência da Computação - Universidade Estadual Paulista "Júlio de Mesquita Filho" (UNESP) - Câmpus de Presidente Prudente
19060-900 - Presidente Prudente - SP - Brasil
gabriel.bevilacqua@unesp.br

Abstract: The use of Artificial Intelligence (AI) in people's daily lives has gained increasing relevance with the advancement of Natural Language Processing (NLP) techniques. Among all applications include text summarization and image generation based on information extracted from news across various online sources, enabled by Large Language Models (LLMs). However, challenges remain regarding text quality, objective evaluation, and redundancy. The work carried out in this project involved developing and evaluating an automatic summarization extractive and a model using LLM and generating mind maps.

Resumo: O uso de Inteligência Artificial (IA) no uso diário das pessoas tem ganhado cada vez mais relevância desde o impulsionamento das técnicas de Processamento de Linguagem Natural (*NLP*). Entre suas aplicações destacam-se o resumo e a construção de imagens baseado em informações retiradas de notícias dos mais diversos veículos disponíveis na internet possibilitado pelo uso de Linguagem de Larga Escala. Todavia, existem desafios como qualidade textual, avaliação objetivas e redundâncias de texto. O trabalho feito nesse projeto foi de desenvolver e avaliar um modelo de resumo automático extrativo e um modelo utilizando *LLM* e confecção de mapas mentais.

1. Introdução

Devido ao grande número de informações digitais, em especial em notícias onde vários veículos de notícias mundo afora publicam vários textos jornalísticos dos mais diversos assuntos, os usuários podem ter desinteresse ao ler dependendo da quantidade de texto envolvido na matéria, no qual se pode perder a vontade de ler e perder o foco. Outro ponto é que o usuário pode procurar via informação de terceiros, que ao lerem a notícia, podem fazer resumos sem uma regra ou apenas fazer uma sumarização pobre de informação.

A percepção desses problemas, junto com a dificuldade de organizar as ideias, principalmente para pessoas que consomem muito conteúdo por dia, evidenciou a necessidade por uma solução eficaz na extração de informações importantes. Isso combinou com a criação com um projeto que envolve algo muito utilizado hoje em dia que são as Inteligências artificiais (IA) que podem a partir de um treinamento fazer esses resumos de textos graças a aplicação técnica de Processamento Natural de Linguagem (PNL), utilizando adicionalmente de modo extrativo métodos de

representação visual em mapas mentais a intenção é facilitar diminuir a barreira para o ganho de conhecimento sobre um texto.

O problema central deste projeto se baseia em aplicar a sumarização de textos de maneira que se consiga utilizar algumas métricas e outros artefatos nos quais podem ser úteis na maneira de compreensão de notícias, tendo assim maior facilidade o acesso a informações relevantes de modo que as partes principais dos conteúdos fiquem retidos para o leitor.

O objetivo geral se baseia em desenvolver uma solução baseada em modelagem de IA com PLN para resumo de textos jornalísticos nos qual a intenção é estudar resumos abstratos e extrativos, além de organizar o texto com o apoio de mapas mentais, que serão feitos por meio de imagens.

2. Referencial Teórico

2.1 Conceitos base de Inteligência Artificial aplicado ao projeto

A inteligência Artificial é uma área vasta dentro da computação, ela engloba outros conceitos como o de *machine learning, deep learning e LLMs*, sendo essa última o que foi mais amplamente estudado no projeto (TSIRMPAS et al., 2024). Dentro dessa parte desta subparte de IA foi operado estudo focado em linguagem humana, chamada de Processamento de Linguagem Natural, área no qual especializado na língua inglesa, em textos de conteúdo jornalísticos de grandes portais de notícias como o da *CNN* e DailyMail (HERMANN et al., 2015).

Através do grande volume de textos de datasets existe uma parte mostra os *highlights*, parte onde mostra os destaques das notícias de maneira enxuta, dessa maneira através do treinamento, a Inteligência Artificial começa a aprender de maneira não supervisionada, que é um uso prático de *Machine Learning*. Através de vários exemplos, ele começa e entender as estruturas e os pontos importantes quando comparado uma notícia a outra e observando os pontos principais, o que dessa maneira formam as redes neurais para fazer sumarização (SUTSKEVER et al., 2014).

Isso nada mais é do que *LLM*, pois faz com que o sistema compreenda e gere textos baseado em aprendizado de máquina e redes neurais. Hoje em dia existem vários *LLMs* para diversos tipos, onde alguns são de uso geral como o *Claude (Anthropic)*; outros mais específicos como o *MedicSearch*; outros modelos que funcionam de maneira multimodais como o *DALL-E (OpenAI)*; e de maneira Compacta como o *LLaMA 2 (Meta)*. No caso desse projeto por mim realizado é uma *LLM* especializada em sumarização utilizando métricas já previamente estudadas.

2.2 Tipos de Sumarização e Teoria

A sumarização automática é uma técnica que busca reduzir o material apresentado no texto de maneira que se preservem informações importantes para o usuário final. Dessa forma, dentro das possibilidades existentes atualmente, temos hoje dois tipos de resumo em essência: o método extrativo e o método abstrato (NENKOVA; MCKEOWN, 2012).

2.2.1 Método Extrativo

A Sumarização do tipo extrativa é um tipo de resumo no qual se utiliza o texto original para fazer o resumo, ou seja, extrair sentenças mais importantes de modo direto do texto e sem modificar sua estrutura. Para esse tipo de tarefa, usam-se algumas técnicas que são necessárias para manter a organização e a coerência do texto.

2.2.1.1 TF-IDF

Uma das técnicas é a análise de frequência de termo (*TF-IDF*), onde avalia a importância de uma palavra dentro de um documento dentro de um grupo de documentos. É feita uma conta no qual usa a frequência de uma palavra e a frequência inversa de documentos (RANI; BIDHAN, 2021), ou seja, penaliza termos que aparecem em muitos documentos, como por exemplo artigos.

$$TF\text{-}IDF(t,d,D) = TF(t,d) \times IDF(t,D)$$

Ajustando o algoritmo *TF-IDF* para que leve em consideração a sentença no texto, fazendo as mesmas pontuações termo a termo e assim extraindo frases melhor pontuadas. É importante ressaltar que, para que esse método seja aplicado será necessário realizar um pré processamento contendo tokenização, *stop-words* e lematização.

Apesar de sua facilidade de implementação e não há consumação substancial de poder computacional, ele tende a ignorar partes como a semântica, privilegiar palavras raras e tirar frases que mesmo que, mesmo que sejam importantes, utilizam termos considerados básicos demais pelo algoritmo.

2.2.1.2 Text-Rank

O método *Text-Rank* também foi utilizado, de modo que se observa as sentenças como se fossem nós dentro de uma rede, semelhante a uma estrutura de dados do tipo grafo (BONDY; MURTY, 1976). As conexões são similaridades semânticas entre elas que são calculadas via sobreposição de palavras e pela similaridade cosseno.

$$Cosine\ Similarity = (S_1 * S_2 / ||S_1| * |S_2||)$$

Onde S_1 e S_2 são vetores que representam a sentença

Enquanto o *TF-IDF* foca principalmente na importância termo a termo de cada sentença, o *TextRank* se preocupa com a relação estrutura global do documento (RANI; BIDHAN, 2021). O intuito é buscar informação e estruturas centrais, de modo que seu produto seja um resumo com uma seleção bem formada.

2.2.2 Método Abstrato

O método de Sumarização Abstrata representa uma abordagem de maneira que não se retira sentenças e palavras de modo “frio”, ou seja, diretamente do texto, mas sim que maneira de como os humanos podem fazer, aprendendo a arquitetura de texto do tipo exemplo, o significado essencial do documento original e utilizando termos do texto apenas utilizando sua maneira de organizar o pensamento IŞIKDEMIR (2024).

Aqui é utilizado o campo não somente sintático de uma linguagem, mas também o campo semântico com todas suas paráfrases e linguagem cotidiana.

2.2.2.1 Arquitetura

Para esse tipo de trabalho é utilizado uma arquitetura específica de *Encoder-Decoder* (codificador-decodificador) com modelos de *Seq2Seq* (sequência-para-sequência), onde sua função é a pedra base de construção para a sumarização modernizada (SUTSKEVER et al., 2014). Nesse tipo de estrutura, o codificador é responsável pelo processamento *token* a *token*, convertendo-o em vetores e capturando as relações semânticas do documento original, bem como o seu contexto.

A parte do decodificador se responsabiliza principalmente pela geração de resumo de maneira auto-regressiva , funcionando como se fosse um tradutor da versão original para uma versão mais enxuta do texto.

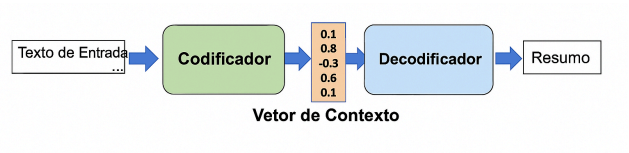

Figura 1: Arquitetura Encoder Decoder
Fonte: Adaptado de Medium, *Exploring the Power of Encoder-Decoder Models: Pros, Cons, and Applications*, 2023

A grande vantagem dos modelos *Seq2Seq* está na sua capacidade de lidar com PLN, de modo que seu uso com deep learning se torna próprio para sumarização de textos, bem como outros usados como a tradução de texto.

2.2.2.2 Modelo para Fine Tuning

Produzido pela *Google AI*, os modelos *T5 (Text-to-Text Transfer Transformer)*, são modelos disponibilizados que utilizam a arquitetura encoder-decoder para fazer treinamento de IA para solucionar problemas envolvendo o campo textual (RAFFEL et al., 2020). A princípio todos os cinco tipos de modelos (*small, base, large, 3b e 11b*) são modelos pré-treinados, portanto, para fazer o ajuste fino (*fine tuning*) se torna uma tarefa mais simplificada do que treinar um modelo todo do zero.

O modelo pré-treinado é utilizado especificamente para sumarização, utilizando como base pares de resumos e documentos como alvo de referência, além de ter seus parâmetros serem atualizados usando gradiente descendente para minimização da função de perda.

Como objeto de estudo foi utilizado dois modelos, o primeiro é o *T5-small* e o *T5-base*, sendo a versão final do projeto utilizando o modelo *T5-base*, devido a sua maior gama de parâmetros tanto para codificar quanto para decodificador, cerca de 3x maior do que o modelo *small*. O desempenho do base é mais equilibrado e não consome tanto consumo computacional quanto os outros modelos maiores.

Hanif (2023) comparou os modelo *T5* contra outros modelos como o *BERT*, *GPT2* e *BART* e observou que ele (*T5*) teve um desempenho melhor usando seus parâmetros, em especial em métricas *ROUGE-1, ROUGE-1 e ROUGE-L*. Em outro artigo feito por Dharrao et al. (2024), observou-se que em *ROUGE-1* e *Meteor* o *T5* teve melhor desempenho do que outros modelos, dessa forma a escolha do modelo *T5-base* serve

Explorando mais seu paradigma, o *Text-to-Text* funciona como um simples modelo base da computação, em especial o Entrada-Processamento-Saída (EPS), onde a entrada é o texto original, o processamento seria a série de cálculos e técnicas dentro do código para a sumarização, e por fim a saída seria o resumo abstrato.

2.2.2.3 Tokenização e Codificação textual

Para a Tokenização do modelo abstrato optou-se pela própria tokenização dada pela biblioteca *T5*, baseado em técnica de subpalavras, que pode controlar maior vocabulário aberto e palavras menos comuns, de modo que se tenha domínio dessa etapa.

Dessa forma, além de "falar a mesma língua" do *T5*, esse tipo de tokenização ganha muito enfoque devido às grandes possibilidades de aprofundamento tanto na parte de prefixos, sufixos e na parte do próprio radical.

Nos termos de normalização de sequência foi utilizada duas técnicas em específico, são elas a de *Padding*, onde adiciona *tokens* escritos como *<pad>* até atingir índices pré definidos de *Max Source* e *Max Target*, de modo que se padronize os lotes de entrada.

Outra técnica está ligada a evitar sobrecargas de memória, bem como a inconsistência de *batch*, é a técnica de *Truncation*, que corta textos que excedem o limite máximo, melhorando assim a estabilidade do treinamento e evitando inconsistência de *batch*.

2.2.2.4 Estratégias de Geração

Como modelos baseados em *Transformer*, a geração é feita de modo auto-regressiva, ou seja, o modelo prevê a próximo token com base no contexto anterior repetindo o processo até se encontrar com os limites previamente impostos. Todavia a estratégia de seleção é variada e precisa de precisão determinística e linguística.

A técnica de *greedy search* seleciona o token de maior probabilidade a cada passo, mas sozinho ele pode gerar textos repetitivos e de baixa diversidade de palavras novas.

Beam Search foi uma das estratégias usadas que mantém de maneira paralela as sequências mais prováveis de *tokens* que vão surgir à saída, ou seja, ela não somente prevê as sequências mais prováveis, mas tenta outras hipóteses. Esse procedimento se mantém até o término do comprimento do texto.

Outro método foi o de *length penalty*, onde durante a sequência da função anterior (*beam search*), ela ajusta a pontuação de modo que ele normaliza o comprimento da sequência, favorecendo desse modo saídas mais íntegras. Quando o tamanho do parâmetro da penalização for maior do que 1 favorece a sequência mais longa, caso contrário favorece mais curta e se por acaso for igual favorece uma sequência mais neutra.

$$Score = (log\ prob)\ /\ (tamanho\ \ do\ parâmetro\ da\ penalização)$$

Para o encerramento é utilizado o *early stopping*, uma alternativa dentro do *beam search* que ajuda a compactar solução, de modo que uma solução que teria geração de *tokens* até o seu máximo poderia gerar informações desconexas e sem eficiência adequada, assim esse método verifica todas as possibilidades de escrita sejam encontradas, ao invés de completar até o fim.

2.2.2.5 Otimização e Fine-Tuning

O *fine-tuning* adapta o modelo pré-treinado para que ele possa ser usado para uma tarefa em específico, dessa forma durante o projeto foi utilizado de maneira especial para a sumarização de textos. Porém para que fosse possível utilizar essa técnica foi necessário utilizar alguns métodos especiais.

Um deles foi o *learning rate*, que é responsável por monitorar e direcionar as atualizações dos pesos durante o treinamento. No caso da arquitetura *Transformers* é necessário ter taxas menores para a preservação de conhecimento do modelo pré-treinado, cerca de 1e-5 até 3e-5 (DEVLIN et al., 2019).

Outro é o Scheduler que junto com *learning rate* ajusta de maneira dinâmica a taxa ao longo do treinamento. Outros valores já pré definidos como o *warmup*, aumentam ainda mais a estabilidade durante o treinamento.

Gradient accumulation é responsável principalmente pela simulação de batches maiores, uma vez que essa simulação ajuda a simular *batches* grandes e hardware limitado, como foi o caso. Dessa forma mais dinamicamente acumulando gradientes de múltiplos mini-batches antes de atualizar pesos.

$$Batch\ Efetivo\ \ =\ \ batch\ size\ /\ passos\ de\ gradiente\ de\ acumulação$$

Outro método para facilitar o processamento dentro do ambiente de execução é o de *mixed precision training*, dessa forma ele consegue reduzir o consumo de memória e acelerar a computação da *GPU*, dessa forma ele altera entre números de 16 *bits* e 32 *bits*, evitando casos de *underflow*.

Por fim outros métodos também são valiosos para a regularização como o *gradiente clipping* que previne erros de estouros durante o *backpropagation*, o *weight decay* que ajuda na penalidade de pesos grandes que evitar *overfitting* e *label smoothing* que distribui a probabilidade entre todos os *tokens* ao invés de focar em especificamente em um, prevenindo *overconfidence*.

2.3 Métricas

Um desafio no qual podemos ter está na parte de avaliação do resumo, uma vez que podemos ter múltiplos resumos válidos para um mesmo texto base, dependo do indivíduo e de sua maneira de organizar as ideias é possível ter resumos dos mais diversos tipos de estrutura e observando pontos que ressaltam mais o pensamento em uma pessoas do que para outras.

Devido a tamanha complexidade do tema, as métricas utilizadas no projeto são métricas flexíveis que podem ajudar na questão da qualidade da sumarização, durante o projeto foram utilizadas as métricas *ROUGE, Attractiveness Score* e *Compression Ratio.*

2.3.1 ROUGE (Reccal-Oriented Understudy for Gisting Evaluation)

O *ROUGE* é uma métrica utilizada para comparar um resumo gerado por IA comparado a um ou mais resumos de referência por de um resumo gerado por máquina (Lin, 2004), ou seja, a qualidade do resumo de máquina tem como princípio a comparação a um feito por um humano. Essa métrica é composta por uma família de três tipos, que se consegue distinguir de acordo com sobreposição de n-gramas.

O *ROUGE-1* utiliza palavras individuais (ou também chamadas de unigramas), no qual observa as palavras mais importantes no texto e a frequência no qual ela aparece, dessa forma ele verifica a similaridade do resumo humano com o de máquina. Em questão de contar primeiro se mede o *recall*, após isso *precision* e por fim o *score*.

$$Recall = (unigramas\ em\ comum)\ /\ (unigramas\ na\ referência)$$
$$Precision = (unigramas\ em\ comum)\ /\ (unigramas\ no\ resumo\ gerado)$$
$$Score = 2\ *\ (precision\ *\ Recall)\ /\ (Precision\ +\ Recall)$$

O *ROUGE-2* utiliza bigramas, ou seja, palavras consecutivas, de modo que trabalha de maneira similar ao primeiro da família, porém o *ROUGE 2* se preocupa mais com a fluência, ordem e contexto ao resumo. Também como no *ROUGE-1* as contas precisas para fazer a pontuação final são as mesmas, apenas mudando a unidade unigrama para bigrama

Por fim o *ROUGE-L* se baseia em *longest common sequence (LCS),* ele usa maior sequência de palavras no texto, de modo que se preocupa mais com a parte da precisão, sem se preocupar com uma sequência exata sequencial, ou seja, ele não usa como parâmetro necessariamente frases com sequências exatas de palavras, como usa o *ROUGE-N*, por exemplo. A princípio a conta se modifica da seguinte maneira, ao invés de unigran ou bigramas é utilizado o *LCS(X,Y)*, onde X é o do resumo gerado e Y do resumo humano, dessa forma ele se divide o comprimento primeiramente da referência para o *Recall* e do gerado para a Precisão.

$$Recall = LCS(X,Y)\ /\ comprimento(referência)$$
$$Precision = LCS(X,Y)\ /\ comprimento(gerado)$$
$$Score = 2\ *\ (Precision\ *\ Recall)\ /\ (Precision\ +\ Recall)$$

Apesar de ter uma boa métrica o *ROUGE* apresenta limitações na parte de semântica mais profunda e se foca na correspondência de palavras entre um texto e outro, e não avaliam coerência ou gramaticalidade do texto (Lin, 2004).

2.3.2 Abstractiveness Score

Com o intuito de atingir textos mais abstratos, se usou a métrica de *abstract score*, que nada mais é do que leva em conta a cobertura e a densidade de palavras e fragmentos, respectivamente, para verificar o grau de abstração de um texto gerado sem copiar o texto de referência (GRUSKY et al., 2018). Aqui se vê uma divisão de águas na parte da distinção de resumos extrativos de abstratos, de modo que se observa

mudança de palavras e reformulação das frases sem mudar o sentido do texto em questão.

No caso seu sistema de conta se baseia 2 ou 3 gramas, e conta quantos desses n-gramas não aparecem no resumo final gerado pela IA, e por fim se divide pelo número total de n-gramas do resumo.

$$Abstractiveness = 1 - (n - gramas\ no\ resumo \cap n - gramas\ no\ documento)\ /\ (total\ de\ n - gramas\ no\ resumo)$$

Quanto mais próximo de 1 maior será o grau de abstração do texto gerado. No entanto quanto menor o valor mais extrativo se torna o resumo gerado pela máquina, dessa forma é possível observar. Mas é sempre importante observar no texto se ele não gera resumos que apesar de serem altamente abstratos não sejam relevantes como resumo, por isso é importante o uso de outras métricas.

2.3.3 Compression Ratio

Usado principalmente para monitorar a condensação do resumo, o *compression ratio* de modo simplificado funciona como um compactador, como se fosse alguns softwares de compactação de arquivos como o *WinRar*, porém funciona em questão de palavras e sentenças. Pode-se descrever sua fórmula como:

Compression Ratio = len(resumo) / len(documento)

Uma compressão que dê por exemplo um valor de 0.4, inside que foi resumido 40% do texto, isso pode ser mudado de acordo com a modelagem. Importante dizer que ele não é uma métrica de qualidade, de modo que precisa ser acompanhado por outras métricas.

2.4 Estruturação de Conhecimento em Mapas Mentais

Os mapas mentais, ou árvores de conhecimento, que foram conceitualizados por Tony Buzan na década de 1970 baseiam-se essencialmente na representação visual de informações que se relaciona ao modo de pensar (BUZAN; BUZAN, 2006), partindo de um ponto central e gerando ramificações de conhecimento sobre um determinado assunto.

Usando conceitos previamente estudados na parte de resumo de texto, em especial o PLN que é fundamental na hora de organizar pensamentos, outros conceitos como as palavras funcionais de alta frequência, os *stopwords*, também são tratados para condensar melhor o conhecimento.

Para que fosse possível fazer a categorização de palavras é necessário utilizar a técnica de *POS tagging (Parts-of-Speech Tagging)*, dessa forma categorizando cada palavra (TOUTANOVA et al., 2003) dentro do texto já mais enxuto. Nessa técnica os substantivos *(NN, NNS, NNP e NNPS)* representam entidades e conceitos principais, enquanto os verbos *(VB, VBD, VBG, VBN, VBP e VBZ)* indicam as ações presentes no texto, já os advérbios *(RB, RBR e RBS)* modificamos verbos e adjetivos, outro exemplos como numerais e adjetivos também precisam ser considerados.

Para efeitos de visualização real foi necessário utilizar a estrutura de dados de grafos (BONDY; MURTY, 1976), nos quais frases e palavras chaves poderiam ser representados como vértices (ou nós) sendo o principal dele a vértice central, e as

arestas que correlacionam as frases e conceitos através de hierarquia, regidas também por um nível de profundidade entre elas.

Para que ocorresse a extração é necessário fundir técnicas, entre elas a de filtragem por categoria gramatical, o comprimento mínimo de palavras, que no caso foi de 3 e a remoção de *stopwords*, no caso de critérios estatísticos foi implementada a parte de frequência implícita e sistemas de pesos para cada categoria gramatical.

Por último é importante evitar redundâncias no texto, o programa usa o conceito de mecanismo de rastreamento baseado em conjuntos (CARBONELL; GOLDSTEIN, 1998), no qual mantém o registro de todas as palavras e frases já extraídas, desse modo concedendo uma melhor eficiência das informações dadas pelo mapa mental.

3. Aplicação Técnica e Metodologia

3.1 Visão Geral da Abordagem Proposta

O trabalho propôs o estudo e o desenvolvimento de sistemas de sumarização extrativa e abstrata, bem como sua avaliação, com a junção de técnicas para cada tipo de resumo. A abordagem teve como alicerce a seguinte estrutura: seleção do *dataset*; implementação das técnicas de resumo extrativo; implementação das técnicas de Resumo Abstrato usando *fine-tuning*, avaliação quantitativa dos resumos e o resultado que foi dado pelas métricas; implementação de mapas mentais; construção da interface gráfica; análise comparativa e observação dos resultado por seres humanos.

3.2 Escolha de Dataset

A partir da busca de *datasets* que pudessem suprir as necessidades apresentadas pelo problema a ser estudado no projeto surgiu alguns questionamentos, uma delas foi que idioma e de quais veículos de empresa obter e até mesmo qual seria o melhor formato de arquivo de dados trabalhar. Dessa maneira foi escolhido datasets de língua inglesa (en-US e en-GB, respectivamente inglês padrão dos EUA e do Reino Unido), devido sua maior acessibilidade de se encontrar na *internet*.

Os veículos de comunicação contidos no dataset eram bem grandes e por isso foi levado muito em consideração e o formato do arquivo foi do tipo *parquet*, que é um formato de código aberto baseado armazenamento em coluna com o objetivo de ser mais eficiente em tamanho do que arquivos *CSV*.

Dessa maneira o Dataset escolhido foi o "cnn daily mail" Hermann et al. (2015), hospedado na plataforma *Hugging-Face* que é especializado em modelagem e datasets para IA. O Dataset é bem simples com apenas três colunas chaves, a do *id*, *article* e *highlights*, contendo hoje três versões disponíveis sendo a usada no trabalho a versão 3.0.0, que é especializada para resumos abstratos e extrativos.

3.3 Implementação da Sumarização Abstrata

De acordo com o material teórico obtido através da seção 2.2.2, conseguiu-se desenvolver uma modelagem de IA no qual utilizando o modelo *T5-base*, pode-se aplicar na prática os conhecimentos e configurações específicas para sumarização abstrata.

3.3.1 Seleção e Preparação do Dataset

Como já mostrado anteriormente, o dataset utilizado foi o *CNN/DailyMail 3.0.0*, apesar de sua versão original ter cerca de 287.113 exemplos para fazer o treinamento, observou-se que durante os treinamentos uma grande heterogeneidade na qualidade, fazendo com que os resumos ficassem de maneira muito excessivamente extrativos, quase cópias literais do texto, dessa maneira precisou-se fazer uma filtragem para que tivesse resumos mais abstratos.

Para garantir a filtragem foi utilizados algumas técnicas, entre elas os artigos deveriam conter entre 150 e 600 palavras, tirando dessa forma artigos muito curtos e muito longos, o limite do *T5* também foi levado em consideração que são 512 *tokens* principalmente para um resumo possivelmente muito comprido, que poderia prejudicar a sumarização.

Definiu-se que as sumarização deveriam ser entre 20-55 palavras, onde se pode ver em resumos muito pequenos, por serem muito fragmentados e muitos longos que poderiam ser demasiadamente grandes quando observado até mesmo ao exemplo do dataset. Como dito anteriormente uma das técnicas foi a de compressão, ou seja, o *compression ratio*, que foi de 0.04 e 0.08, algo entre 4% e 8% do texto.

Um outro critério foi a de sobreposição de trigramas, onde se a sobreposição fosse superior a 50% ela seria considerada extrativo demais para uma sumarização, pois basicamente teríamos cópia de trechos ao invés de parafrasear.

A diversidade lexical também foi levada em consideração.

Unique Ratio = palavras únicas / palavras totais

A intenção era de se ter maior gama de possibilidades de parafraseamento.

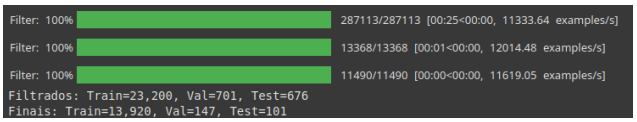

Figura 2: Dados usados para o treinamento
Fonte: Código desenvolvido no Google Colab

Dentro da grande quantidade de exemplos, a filtragem se deu devido ao tipo de método usado, o *fine-tuning*, a capacidade computacional e a qualidade do resumo.

3.3.2 Tokenização e Pré Processamento

O *T5TokenizarFast* que foi utilizado, foi utilizado a implementação *SentencePiece*. Devido a especificação do *T5* (RAFFEL et al., 2020), todas as entradas recebem um prefixo que especifica a tarefa, dessa forma foi usado para fazer a sumarização, além disso casos como a quebra de linhas, maximização de *tokens* fonte, maximização de tokens alvos forma melhor implementados, dessa forma utilizamos o próprio modelo para fazer a tokenização e pré-processamento.

3.3.3 Carregamento e Configuração

Utilizando o *T5-base* de 222.903.552 parâmetros foi utilizado devido ao seu custo-benefício, onde outros modelos mais robustos poderiam sofrer problemas devido não só a *GPU*, mas como também ao limite de *VRAM* disponível no *Google Colab*.

3.3.4 Configuração e Treinamento e Otimização

Dentre os hiperparâmetros apresentados foram definido o learning rate de 2e-5, valor dentro do que foi definido na literatura, dentro dos testes feitos foi estabelecido nessa caso junto com dataset filtrado 12 épocas de treinamento. Alguns parâmetros foram com *warmup* de 15% e batch size físico de apenas 2 exemplos por dispositivo, todavia para compensar esse *batch* implementou o *gradient accumulation* com 8 *steps*, resultando em um batch específico de 16 exemplos.

No caso para se reduzir o *overfitting weight decay* de 0.02, além de penalização L2 sobre pesos maiores, *label smoothing* de 0.1 que distribui a probabilidade entre os *tokens* prevenindo *overconfidence*, um *gradient clipping* de 1.0 que limita uma grande quantidade de gradiente e com isso uma explosão do *backpropagation*.

Crucialmente para a viabilidade do projeto no treinamento uma *mixed precision training* de 16 *bits*, podendo ser alternado para 32 bits, para consumir menos memória e ajudar no treinamento da *GPU*. O modelo era treinado num *step* de 200 passos, principalmente para se conseguir ver melhor a evolução do modelo, em especial o *ROUGE* e a abstração, além disso como medida preventiva foi implementado um algoritmo que salvava os dois melhores passos do modelo, além disso deletava as *steps* que tinham menor desempenho que a melhor passo do treinamento, pois a cada passo gerava cerca de 1.8 *Gb* em arquivos.

3.3.5 Implementação da Geração de Sumários

Para a geração de sumário implementou-se uma função que encapsula o processo de geração, com 512 *tokens* para a tokenização, coloca-se o prefixo "*summarize*", e é utilizado o truncation quando necessário, e os tensores são transferidos para a *GPU*. Para a geração é utilizada um *beam search* de 4, os *tokens* feitos na sumarização estão entre 30 e 150, é implementado também para evitar repetição de 3 *tokens* consecutivos.

Early stopping permite encerrar assim que 4 hipóteses foram encontradas e desativando o do *sample*, dessa forma desabilita a amostragem estocástica.

3.4 Implementação da Sumarização Extrativa

Como possibilidade comparativa de resumo automático em relação ao modelo feito em *T5*, desenvolveu-se um sistema de sumarização extrativa. O *pipeline* de implementação se desenvolveu da seguinte maneira:

3.4.1 Pré Processamento

Antes de desenvolver qualquer classe de resumo foi necessário desenvolver uma técnica de limpeza do texto, para tirar possíveis ruídos que podem ocorrer, entre eles a remoção de possíveis *URLs*, a tokenização das sentenças via a biblioteca de tokenização *NLTK* que também remove as stop-words que poderiam ter no texto.

Além disso, é importante notificar que a tokenização foi baseada na linguagem inglesa, a mesma do *dataset*, além de poder contribuir com algumas particularidades da linguagem como suas abreviações. Por fim como decisão de *design* foi retirado caracteres especiais (@,#,^) mas o mantimento de caracteres de pontuação tradicionais.

3.4.2 Vetorização TF-IDF

Para a vetorização escolheu o padrão de unigramas, devido a maior facilidade e uma possível mudança para bigramas e trigramas aumentaria ainda mais a complexidade do código.

3.4.3 Matriz de Similaridade e PageRank adaptado

A matriz de similaridade além de sua eficiência em questão de complexidade (O(n)), ela é uma base importante para o código devido que podemos fazer comparação entre frases, no código em questão apenas se removeu a auto-similaridade.

Para o *Page-rank* foi utilizado as devidas características: damping de (0.85) que é o valor usado de maneira padrão (BRIN; PAGE, 1998), e a interatividade foi de 10, para que se consiga uma convergência satisfatória.

3.4.4 Heurísticas complementares

Em relação a bônus foram implementadas duas, ambas do tipo multiplicativo. O primeiro em relação a posição, que melhor se adapta e texto jornalísticos, onde o decŕecimo linear de 1.3 para 0.9 foi em decorrência de ajustes com o passar dos testes. Outro bônus foi o de entidade, em especial para *NNP* e *NNPS*, de modo que eles não dominem o resumo mas tenha grande relevância, por isso a variação entre 1.2 e 1.0.

Em relação a parâmetros foram configurados para o número mínimo de sentenças entre 2-3 (para extrair o mínimo informativo), o máximo entre 4-5 sentenças, em relação aos caracteres no testes resumos de 150 caracteres igual ao abstrato teve uma queda em questão de informação dada, dessa forma decidiu-se aumentar para 250 caracteres.

Na parte de seleção foi implementado que o algoritmo obtivesse a sentença de maior valor entre as duas primeiras sentenças, para as demais foi feito uma solução gulosa, demo que seleciona por score decrescente. Outras partes do código se preocuparam pela reordenação cronológica e parada antecipada em relação aos limites de número de sentença e caracteres.

3.4.5 Pós-Processamento

Por fim, o código na sua fase de pós-processamento se preocupa em detalhes mínimos, como a normalização de pontuação, remoção de sentenças com menos de 5 *tokens*, a menos que seja a última do resumo e por fim a correção de espaçamento.

3.5 Geração Mapas Mentais

O fluxo principal é muito parecido com o de sumarização, onde o *input* é o resumo já feito, o processamento de tokenização e filtração e o *output* sendo a geração do mapa mental. Primeiramente é necessário passar pelo processo de tokenização, a categorização via *POS tagging*, a extração de substantivos próprios, a contagem de frequência, a seleção dos substantivos mais frequentes e por fim a saída do mapa mental.

O rastreamento das palavras que garante que nenhuma informação não redundante é dada pela linha abaixo

used_words =set()

Para o processamento iterativo de sentenças o algoritmo processa cada uma individualmente, atribuindo a interação do nó que pode ser tingida de verde, restringindo o nó visual com apenas 65 caracteres e espessura de 4 pixels

Em parte de pesos para cada tipo de categoria foi implementado da seguinte forma:

- Números (*CD*): peso 10;
- Substantivos próprios (*NNP/NNPS*): peso 9;
- Bigramas: peso 8;
- Verbos: 6;
- Advérbios: peso 4;
- Adjetivos: peso 5;
- Substantivos comuns (*NN/NNS*): peso 7.

Para a renderização final usou o algoritmo *Force-Directed-Placement* do *Graphviz* que calcula de forma otimizada as posições de cada nó e evitando sobreposições, alguns parâmetros também foram colocados como *overlap=false* para evitar sobreposição de elementos e *splines = "curved"* para arestas curvas. Por fim colocou-se o nome do arquivo e o tipo de arquivo a ser salvo na máquina, que no caso o formato escolhido por ser mais simples foi incluído o *PNG*.

3.6 Interface Gráfica

Como o projeto se baseia em uma aplicação *desktop*, o projeto precisou passar por algumas opções de *frameworks* que pudessem ser feito uma interface gráfica para uma melhor experiência do usuário final, dessa maneira dentro das opções disponíveis para a linguagem *Python*, escolhi uma das mais usadas no mercado para esse tipo de tarefa o *Tkinter*.

Podendo ser utilizado nos principais sistemas operacionais presentes hoje em dia para *notebooks* e *desktop* (*Windows, Linux, Unix* e *MacOS*) e de simples uso devido à sua ampla documentação e fácil aprendizagem.

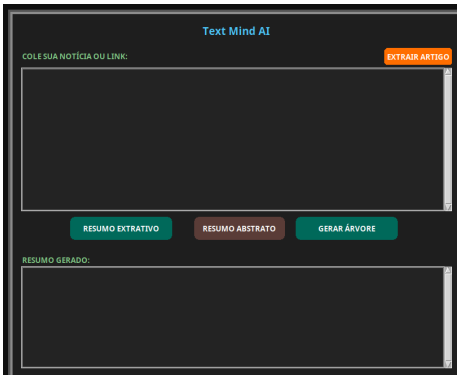

Figura 3: Tabela principal do programa
Fonte: Interface do projeto

Ao abrir o programa o usuário a princípio precisa colar o texto a ser sumarizado ou o link de um artigo, dessa forma caso for um link ele extrai o conteúdo automaticamente. Após isso ele poderá acionar o botão para fazer o resumo abstrato e extrativo, nos dois casos o texto sumarizado apresentará no *Textbox* debaixo, porém caso ele quiser fazer a geração da árvore de conhecimento (Mapa Mental) o sistema fará o download da imagem contendo o mapa.

Apesar do *Framework* não possuir muitas possibilidades de edição de componentes dentro dele, foi muito útil para evitar que se usassem frameworks pagos ou a total migração do projeto para um projeto feito em *Web* baseado em *Django*.

3.7 Ambiente de Desenvolvimento

3.7.1 Hardware

Em termos de *Hardware* para fazer a parte menos robusta da aplicação, que inclui a sumarização extrativa, os mapas mentais e a interface gráfica, foi utilizado um *notebook* com 8GB de *RAM*, processador I7 de 10 geração. Já para o treinamento foi utilizado o serviço do Google Colab, para treinamento foi utilizado as *GPUs L4* e *T4* da *Nvidia*, além de 16 GB de *RAM.*

3.7.2 Software

Para o salvamento dos checkpoints do modelo foram utilizados os serviços de nuvem da *Google*, além do próprio *Colab* para a parte de modelagem, como IDE foi utilizado o *VS Code*.

3.7.3 Bibliotecas e Frameworks

As bibliotecas utilizadas no projeto como um todo foram:

- *Numpy*: Biblioteca fundamental para cálculos numéricos em Python;
- *NLTK*: Tokenização e remoção de Stopwords em textos, em especial para confecção de resumos extrativos;
- *T5 TokenizerFast*: Tokenização nativo do T5, promove estrutura semântica de subpalavras;
- *SkyLearn*: Biblioteca para aprendizado de máquina para o pré-processamento de dados, bem como sua modelagem e avaliação.

- *PyTorch*: Framework feito para Deep Learning, flexivel para criação e suporte nativo para GPU;
- *Transformer*: Framework onde está hospedado os principais modelos, em especial o T5-base utilizado;
- *Newspaper*: Extração de artigos da web;
- *Graphviz: Criação de grafos;*
- *Tkinter*: Framework de interface Gráfica.

Além de outras bibliotecas básicas do *Python* como o *Sys* e o *OS*.

4. Resultados e Observações

Tabela 1: Treinamento do modelo até o passo 1400

Step	Training Loss	Validation Loss	Rouge1	Rouge2	Rougel	Abs
200	3.817000	3.880457	33.490000	13.570000	25.280000	65.920000
400	3.407300	3.429744	34.150000	13.700000	25.770000	64.020000
600	3.168700	3.176380	34.540000	13.930000	26.760000	62.150000
800	3.112200	3.105270	34.290000	14.580000	27.010000	60.940000
1000	3.062100	3.072774	35.510000	15.810000	27.550000	60.420000
1200	3.095300	3.043397	35.630000	15.370000	27.260000	60.150000
1400	3.013700	3.029625	36.200000	16.010000	27.990000	60.180000

Depois de vários testes podemos ver que o modelo conseguiu realizar um resumo abstrato de modelos, apesar de eles muitas vezes optarem por obter outros conteúdos da notícia além do que obtiveram como referência. Pontuação alcançou resultado semelhante a DARAGHMI et al. (2025) nos testes de CNN/DainlyMail com T5 Base.

No exemplo a seguir a uma notícia sobre os estudos sobre a morte de Anne Frank da *CNN* postada em 2015, o mesmo exemplo do resumo extrativo, segundo o código se obteve obter cerca de 29 palavras, 93.8% de compreensão e 20,7% de abstração.

> Referência: Museum: Anne Frank died earlier than previously believed . Researchers re-examined archives and testimonies of survivors . Anne and older sister Margot Frank are believed to have died in February 1945 .
>
> Modelo: Researchers find Anne Frank died a month earlier than originally thought . Researchers re-examine Red Cross archives . Anne Frank was sent to Auschwitz-Birkenau concentration camp in 1944 .

Nesse exemplo o modelo conseguiu realizar uma boa sumarização, obtendo não somente parte importante do conteúdo, como também outros assuntos presentes no texto, porém de maneira mais abstrata.

Outro exemplo foi de uma matéria da *CNN* sobre uma trend no *Twitter* sobre o "*Rex Manning Day*", que reviveu datas de filme comemorados pelo público, o resultado obteve semelhança com o anterior, mantendo algumas informações da referência, mas colocando informações que não foram colocados na referência original, mas também importantes.

Referência: April 8 was huge for "Lost" and "Empire Records" fans . April 14, April 25, October 21 are other big dates in movies .

Modelo: "Rex Manning Day" is celebrated on April 8 . Dates made famous by films include "Miss Congeniality" and "Mean Girls Day"

Mais um exemplo foi uma notícia sobre o Grand Prix da China, na disputa pela liderança.

Referência: Referência: Lewis Hamilton will start on pole in China . Pushed teammate Rosberg into second on the last lap . Rosberg refused to shake Hamilton's hand afterward .

Modelo: Lewis Hamilton beat Nico Rosberg by four hundredths of a second . Ferrari driver Kimi Raikkonen will join Vettel in third place . Hamilton has been quickest all weekend .

Importante ressaltar que dependendo muito de texto a texto, pode acontecer que o modelo não consiga entender o contexto ou abstrair bem o texto. Na avaliação final se obteve os seguintes resultados depois de 3 exemplos.

- ROUGE-1: 32.45%
- ROUGE-2: 13.05%
- ROUGE-L: 25.68%
- Abstração: 63.81%

Mais um exemplo foi a de uma mulher que comeu uma grande quantidade de comida em pouco tempo.

Referência: Molly Schuyler scarfed down three 72-ounce steaks Sunday in Amarillo, Texas .
The Sacramento woman, 35, is a professional on the competitive-eating circuit .

Modelo: Molly Schuyler eats three 72-ounce steaks, baked potatoes, side salads, rolls and shrimp cocktails . She also breaks her own Big Texan record of two 72-oz steaks and sides .

Em relação a resultados do resumo extrativo observou-se que nesse algoritmo o número de caracteres influencia diretamente na quantidade de informação a ser mostrada para o usuário final, num artigo da *CNN* sobre a morte de Anne Frank.

Exemplo com 100 caracteres: Seventy years ago, Anne Frank died of typhus in a Nazi concentration camp at the age of 15. A deadly typhus outbreak caused thousands to die each day.

Exemplo com 250 caracteres: Seventy years ago, Anne Frank died of typhus in a Nazi concentration camp at the age of 15. But new research released by the Anne Frank House shows that Anne and her older sister, Margot Frank, died at least a month earlier than previously thought.

Exemplo com 350 caracteres: Seventy years ago, Anne Frank died of typhus in a Nazi concentration camp at the age of 15. Just two weeks after her supposed death on March 31, 1945, the BergenBelsen concentration camp where she had been imprisoned was liberated timing that showed how close the Jewish diarist had been to surviving the Holocaust.

É evidente que quanto maior o espaço de transcrição maior é a gama de informações feitas pelo algoritmo, além disso a maneira que o texto foi escrito (em questão de conteúdo e a divisão de parágrafos e espaçamento de vírgulas) pode fazer com que o texto se comporte de forma diferente.

Na parte de mapas mentais que usa como base o modelo extrativo, na imagem abaixo mostra o exemplo da seguinte notícia:

> CNN Mercedes driver and F1 championship leader Lewis Hamilton stole pole position for Sundays Chinese Grand Prix from teammate and fierce rival Nico Rosberg in dramatic fashion. Hamilton took first place on the front row on the last lap, beating Rosberg by a slim four hundredths of a second margi.

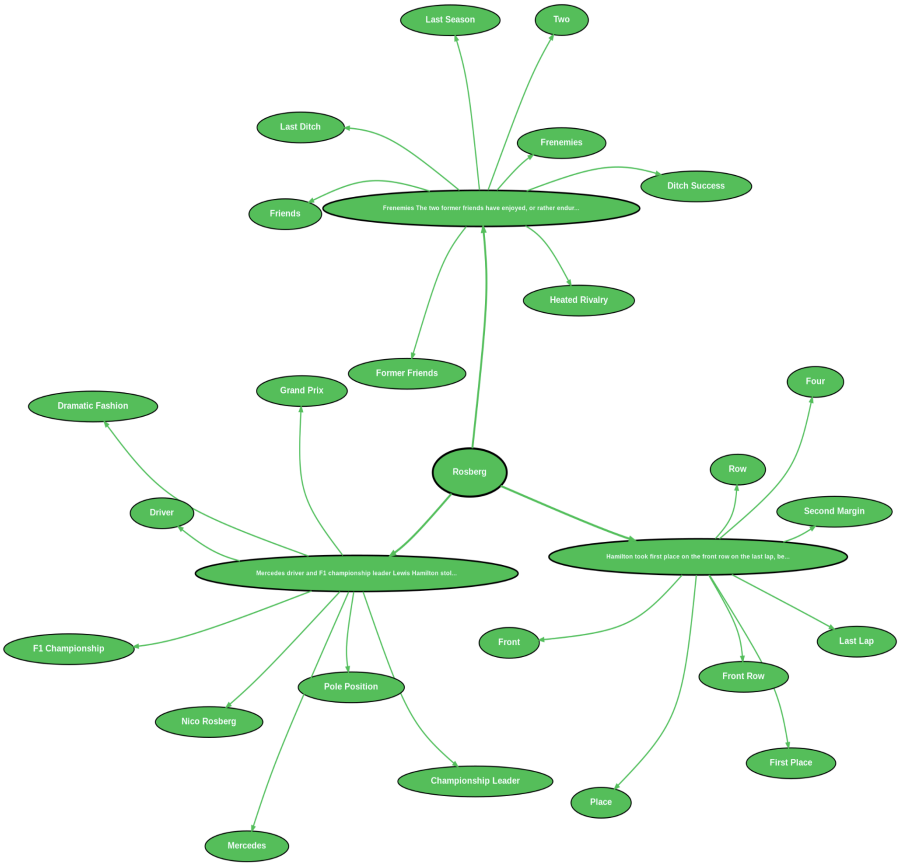

Figura 4: Exemplo de mapa mental
Fonte: Arquivo de png do programa desenvolvido

5. Trabalhos Relacionados

A sumarização automática foi bem abordada com suas diferentes técnicas, onde Nenkova e McKeown (2012) estabeleceram seus principais alicerces, já Lin (2004) conseguiu explorar as métricas com o modelo *ROUGE*. Na parte de sumarização extrativa Rani e Bidhan (2021) conseguiram estabelecer técnicas como *TF-IDF* e *TextRank* como duas das três técnicas de resumo extrativo r Carbonell e Goldstein (1998) propuseram o *MMR* para a melhor reduzir a redundância dos textos.

Na parte de *dataset* Hermann (2015) conseguiu referenciar *CNN/DailyMail* como um dos melhores para a sumarização, enquanto Sutskever e Le (2014) estabeleceram a arquitetura. Para Raffel et al. (2020) conseguiu abordar o *T5*, sendo que Hanif (2023) e Dharrao et al. (2024) conseguiu verificar o modelo contra outros como o *BERT* e o *Pegasus*, sendo que para Hanif observou melhor métrica no *ROUGE* e Dharrao conseguiu ver evolução em outras métricas como o *Meteor*.

Para a filtragem Ravaut et al. (2024) conseguiu verificar a importância dos elementos em relação a sua posição do texto, já Grusky, Naaman e Artiz (2018) estabeleceram para o resumo abstrato os cálculos *abstractness score*.

Para a representação visual Buzan e Buzan (2006) estabeleceram os fundamentos dos mapas mentais, e a implementação em grafos (BONDY;MURTY,1976) e *POS tagging* (TOUTANOVA et al., 2003) para identificar palavras-chaves.

O diferencial desse trabalho foi juntar técnicas tanto de resumo extrativo como o resumo abstrato, além de abordar técnicas de mapas mentais para melhor organização dos pensamentos. Além disso, abordagens como *fine-tunning* e uma implementação com interface gráfica e filtragem rigorosa beneficiaram o modelo e o projeto como um todo.

6. Considerações Finais

O trabalho propôs o desenvolvimento de resumos de texto, em especial jornalísticos e depois sua adaptação para a parte de mapas mentais. Observando as limitações dos dois tipos de sumarização e as plataformas para demonstrar seu conhecimento foi possível demonstrar que os objetivos dados na introdução foram alcançados apesar das limitações computacionais e temporais.

Contudo foi visto que se conseguiu um resultado satisfatório para os dois tipos de sumarização e o emprego básico do uso de mapas mentais utilizando a técnica de grafos, contudo é importante salientar que existe muito espaço para melhorar, seja se baseando por meio de outras técnicas, modelos e/ou materiais.

7. Trabalhos futuros

Como sugestão de trabalhos futuros após esse estudo propõe pesquisar ainda mais a fundo a utilização de modelos com mais parâmetros dentro da família *T5,* utilizar outros *datasets* de notícias com mais diversidade e até mesmo em línguas diferentes,

além de ter um acompanhamento maior com pessoas mais especializadas no estudo de estrutura de textos.

Além de poder utilizar outros modelos concorrentes, em especial o *BERT* e o Pegasus, sendo esse último mais visado de acordo com sua real especialização, pontuação com modelos concorrentes e melhor aprofundamento em questão de aprofundamento de resumo abstrato.

Outra opção interessante seria a extração de textos de notícias em imagens, verificação de fontes e estudar novas técnicas de visualização e extração de informação para melhorar a compreensão dos resumos e imagens geradas. Por fim a correção de erros de código, implementação de novas tecnologias e possíveis refatorações podem ser feitas.

8. Referências

1. IŞIKDEMIR, Yunus Emre. Nlp transformers: Analysis of llms and traditional approaches for enhanced text summarization. **Eskişehir Osmangazi Üniversitesi Mühendislik ve Mimarlık Fakültesi Dergisi**, v. 32, n. 1, p. 1140-1151, 2024. ISSN 2630-5712. Journal of Engineering and Architecture Faculty of Eskisehir Osmangazi University.
2. HERMANN, Karl Moritz et al. Teaching machines to read and comprehend. **Advances in neural information processing systems**, v. 28, 2015.
3. SUTSKEVER, Ilya; VINYALS, Oriol; LE, Quoc V. Sequence to sequence learning with neural networks. **Advances in neural information processing systems**, v. 27, 2014.
4. RANI, Ujjwal; BIDHAN, Karambir. Comparative assessment of extractive summarization: textrank tf-idf and lda. **Journal of scientific research**, v. 65, n. 1, p. 304-311, 2021.
5. NENKOVA, Ani; MCKEOWN, Kathleen. A survey of text summarization techniques. In: **Mining text data**. Boston, MA: Springer US, 2012. p. 43-76.
6. LIN, Chin-Yew. Rouge: A package for automatic evaluation of summaries. In: **Text summarization branches out**. 2004. p. 74-81.
7. RAVAUT, Mathieu et al. On context utilization in summarization with large language models. **arXiv preprint arXiv:2310.10570**, 2023.
8. BUZAN, Tony; BUZAN, Barry. **The mind map book**. Pearson Education, 2006.
9. CARBONELL, Jaime; GOLDSTEIN, Jade. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In: **Proceedings**

of the 21st annual international ACM SIGIR conference on Research and development in information retrieval. 1998. p. 335-336.

10. TOUTANOVA, Kristina et al. Feature-rich part-of-speech tagging with a cyclic dependency network. In: **Proceedings of the 2003 human language technology conference of the north american chapter of the association for computational linguistics**. 2003. p. 252-259.
11. BONDY, John Adrian et al. **Graph theory with applications**. London: Macmillan, 1976.
12. DHARRAO, Deepak et al. Summarizing Business News: Evaluating BART, T5, and PEGASUS for Effective Information Extraction. **Revue d'Intelligence Artificielle**, v. 38, n. 3, 2024.
13. HANIF, Usama. **Research paper summarization using text-to-text transfer transformer (T5) model**. 2024. Tese de Doutorado. Dublin, National College of Ireland.
14. BRIN, Sergey; PAGE, Lawrence. The anatomy of a large-scale hypertextual web search engine. **Computer networks and ISDN systems**, v. 30, n. 1-7, p. 107-117, 1998.
15. TSIRMPAS, Dimitrios et al. Neural natural language processing for long texts: A survey on classification and summarization. **Engineering Applications of Artificial Intelligence**, v. 133, p. 108231, 2024.. ISSN 0952-1976.
16. GRUSKY, Max; NAAMAN, Mor; ARTZI, Yoav. Newsroom: A dataset of 1.3 million summaries with diverse extractive strategies. **arXiv preprint arXiv:1804.11283**, 2018.
17. RAFFEL, Colin et al. Exploring the limits of transfer learning with a unified text-to-text transformer. **Journal of machine learning research**, v. 21, n. 140, p. 1-67, 2020.
18. DARAGHMI, Eman; ATWE, Lour; JABER, Areej. A Comparative Study of PEGASUS, BART, and T5 for Text Summarization Across Diverse Datasets. **Future Internet**, v. 17, n. 9, p. 389, 2025.
19. Figura Arquitetura Encoder Decoder. Imagem Original disponível em: *https://miro.medium.com/v2/resize:fit:720/format:webp/1*aCA6-HH35CY3eYPXXPSHmA.png*